Dart Async, Future & Null Safety Demo

Asynchronous Programming and Null Safety — Assignment Report

Name:	Aayush Kamble
Assignment:	Async API Service (Dart - Future, async/await, Null Safety)
File:	async_api_service.dart
GitHub Repository:	[ADD YOUR GITHUB REPOSITORY LINK HERE]

1. Assignment Completion

This assignment required building a small console-based Dart program that demonstrates asynchronous programming with Future and async/await, together with Dart's sound null safety features. The program simulates an API service that fetches a user profile by ID, modeling realistic network behavior: normal responses, a missing-user (null) response, and two distinct error conditions that must be caught and handled.

All the required functionality was implemented and verified by running the program:

- A UserProfile data model with a nullable field (String? bio) and a required-field constructor, demonstrating Dart's null safety syntax.
- An ApiService.fetchUserProfile() method that is asynchronous (returns Future<UserProfile?>), uses await Future.delayed() to simulate network latency, and returns different results based on the requested user ID.
- Successful fetches for user IDs 1 and 2, including one profile with a real bio and one with a null bio handled via the ?? null-coalescing operator.
- A not-found case (ID 404) where the Future resolves to null instead of throwing, and the caller checks for null before using the result.
- Two thrown-exception cases: a FormatException for an invalid ID (-1) and a TimeoutException for a simulated server error (ID 500), both caught with try/catch inside processUserFetch().

2. Project / Work Quality

The program runs correctly end-to-end (see the terminal output and screenshot in Section 4), and it directly applies the async programming and null safety concepts taught in class:

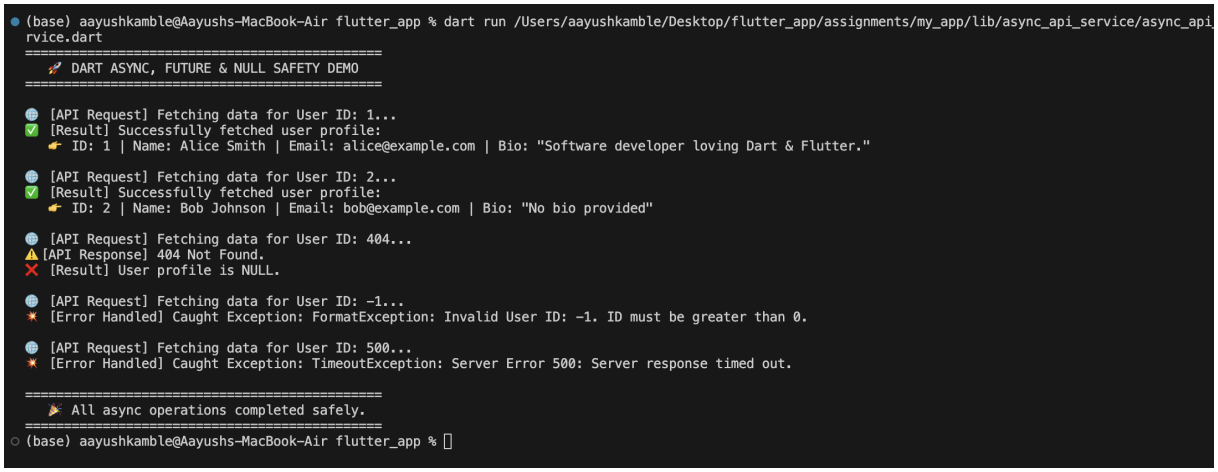
Concept	How it is used in this project
Future & async/await	fetchUserProfile() and processUserFetch() are both declared async and return a Future. Every call site uses await so execution pauses for the simulated network delay without blocking with callbacks.
Nullable types (?)	String? bio on UserProfile and the Future<UserProfile?> return type make it explicit, at compile time, that a bio or an entire profile may legitimately be absent.
Null-coalescing operator (??)	toString() uses bio ?? 'No bio provided' to substitute a safe default the moment a null bio would otherwise be printed, instead of scattering null checks elsewhere.
Null checks before use	processUserFetch() checks if (profile == null) before calling profile.toString(), so a null profile is reported cleanly rather than causing a runtime null-dereference.
try/catch exception handling	Both thrown exception types (FormatException, TimeoutException) are caught in a single catch (e) block, letting one handler cover multiple error types thrown from the same awaited call.

3. GitHub

Repository link: **[ADD YOUR GITHUB REPOSITORY LINK HERE]**. The repository contains the Dart source file (async_api_service.dart), this report, and the terminal output screenshot.

4. Report & Screenshots — Program Output

The screenshot below shows the program being run from the terminal with `dart run`, producing the two successful fetches, the null not-found case for ID 404, and the two caught exceptions for IDs -1 and 500.



Terminal output of async_api_service.dart

5. Source Code — async_api_service.dart

```dart
import 'dart:async';

// ==========================================
// 1. DATA MODEL USING NULL SAFETY
// ==========================================
class UserProfile {
  final int id;
  final String name;
  final String email;
  final String? bio; // Nullable variable (can be null)

  UserProfile({
    required this.id,
    required this.name,
    required this.email,
    this.bio, // Optional parameter
  });

  @override
  String toString() {
    // Using Null-Coalescing Operator (??) to handle null values safely
    String safeBio = bio ?? 'No bio provided';
    return 'ID: $id | Name: $name | Email: $email | Bio: "$safeBio"';
  }
}

// ==========================================
// 2. MOCK API SERVICE (Future, async/await)
// ==========================================
class ApiService {
  /// Simulates an asynchronous API request fetching user data.
  /// Returns a nullable Future<UserProfile?>.
  Future<UserProfile?> fetchUserProfile(int userId) async {
    print('\n[NET] [API Request] Fetching data for User ID: $userId...');

    // Simulate 1 second network latency using Future.delayed
    await Future.delayed(const Duration(seconds: 1));

    // Case 1: Error handling for invalid IDs (Throws FormatException)
    if (userId <= 0) {
      throw FormatException('Invalid User ID: $userId. ID must be greater than 0.');
    }

    // Case 2: Server error scenario (Throws TimeoutException)
    if (userId == 500) {
      throw TimeoutException('Server Error 500: Server response timed out.');
    }

    // Case 3: Null response scenario (User not found)
    if (userId == 404) {
      print('[!] [API Response] 404 Not Found.');
      return null;
    }

    // Case 4: Success - User with full data
    if (userId == 1) {
      return UserProfile(
        id: 1,
        name: 'Alice Smith',
        email: 'alice@example.com',
        bio: 'Software developer loving Dart & Flutter.',
      );
    }

    // Case 5: Success - User with null bio field
    return UserProfile(
      id: userId,
      name: 'Bob Johnson',
      email: 'bob@example.com',
      bio: null, // Null value assigned safely
    );
  }
}

// ==========================================
// 3. HELPER FUNCTION WITH TRY-CATCH & NULL CHECKS
// ==========================================
Future<void> processUserFetch(ApiService apiService, int userId) async {
  try {
```

```
    // Await asynchronous Future call
    UserProfile? profile = await apiService.fetchUserProfile(userId);

    // Null Safety Check using null-aware syntax
    if (profile == null) {
      print('[X] [Result] User profile is NULL.');
    } else {
      // Safe access to properties
      print('[OK] [Result] Successfully fetched user profile:');
      print('   -> ${profile.toString()}');
    }
  } catch (e) {
    // Catching and handling runtime errors/exceptions
    print('[ERR] [Error Handled] Caught Exception: $e');
  }
}

// ==========================================
// 4. MAIN ENTRY POINT
// ==========================================
Future<void> main() async {
  final apiService = ApiService();

  print('================================================');
  print('   [*] DART ASYNC, FUTURE & NULL SAFETY DEMO   ');
  print('================================================');

  // 1. Fetching valid user (with non-null bio)
  await processUserFetch(apiService, 1);

  // 2. Fetching valid user (with null bio)
  await processUserFetch(apiService, 2);

  // 3. Fetching non-existent user (Returns NULL)
  await processUserFetch(apiService, 404);

  // 4. Fetching with invalid ID (Handles FormatException error)
  await processUserFetch(apiService, -1);

  // 5. Fetching with server error ID (Handles TimeoutException error)
  await processUserFetch(apiService, 500);

  print('\n================================================');
  print('   [*] All async operations completed safely.  ');
  print('================================================');
}
```

Note: the emoji used in the original source print statements (globe, checkmark, cross, warning, explosion, pointing hand, rocket, party popper) are shown here as bracketed labels for reliable PDF rendering; the actual .dart file uses the emoji characters visible in the terminal screenshot in Section 4.

6. What I Learned

Understanding Future and async/await

This assignment made the relationship between Future, async, and await concrete for me. Marking fetchUserProfile() as async automatically wraps its return value in a Future, and using await Future.delayed(const Duration(seconds: 1)) inside it lets me pause execution at that exact point without blocking the rest of the program or resorting to nested `.then()` callbacks. Before this assignment I had read about Futures mostly as an abstract “value that will exist later”; writing processUserFetch() and seeing each await genuinely pause for a second in the terminal output made it click that await is really just sequential-looking code hiding asynchronous waiting underneath.

Null Safety as a Compile-Time Guarantee

I learned that Dart's null safety isn't just a naming convention — the compiler actually enforces it. Declaring bio as `String?` instead of `String` meant I could not accidentally use it as if it were always present; I had to either check it or provide a fallback. The `??` operator in toString() (`bio ?? 'No bio provided'`) turned out to be the cleanest way to do this, since it collapses what would otherwise be a multi-line if/else into a single readable expression, applied at exactly the point where the value is used.

Returning null from a Future vs. Throwing an Exception

The most conceptually useful part of this assignment was designing three genuinely different “not a normal success” outcomes and realizing they are not interchangeable. A missing user (ID 404) is an expected, valid outcome, so fetchUserProfile() simply returns null and the caller checks for it with a plain if statement. An invalid ID or a server timeout, by contrast, represents something actually wrong with the request or the server, so those are modeled as thrown exceptions (FormatException, TimeoutException) instead. Writing both paths side by side taught me to ask, for any “failure” case in a real API, whether it is an expected absence of data (model it as null / an Optional-like value) or a genuine error (model it as a thrown exception), rather than defaulting to one approach everywhere.

try/catch Around Awaited Calls

I also learned that wrapping an awaited call in try/catch behaves exactly like wrapping a synchronous call — any exception thrown inside fetchUserProfile(), even though it happens after an asynchronous delay, still surfaces at the await UserProfile? profile = await apiService.fetchUserProfile(userId); line inside the try block, and is caught by the matching catch (e). This was reassuring to confirm by testing it directly: both the FormatException from ID -1 and the TimeoutException from ID 500 were caught by the same single catch block, without needing separate on FormatException / on TimeoutException clauses, since the assignment only required a generic handler that reports whatever went wrong.

Problems Faced and How I Solved Them

- **Deciding where the null check belongs:** I initially considered checking for null inside fetchUserProfile() itself, but that class only knows how to fetch data, not how the caller wants to react to a missing user. Moving the if (profile == null) check into processUserFetch() kept each function responsible for only one concern.

• **Choosing null vs. an exception for the 404 case:** My first instinct was to throw an exception for a not-found user, the same as for the invalid-ID and timeout cases. I changed this after realizing a 404 is a normal, expected response in a real API (the user genuinely doesn't exist), whereas the other two cases represent actual failures — so modeling 404 as a null return instead of an exception better matches how the outcomes are actually used afterward.

• **Catching two different exception types with one handler:** I wasn't sure whether I needed separate catch clauses for FormatException and TimeoutException. Testing showed that a single catch (e) block catches any thrown object regardless of its runtime type, which was sufficient here since both cases only needed to be printed, not handled differently.

• **Confirming the delay actually happens asynchronously:** To be sure await Future.delayed() was really pausing execution and not just being skipped, I watched the terminal output appear with a visible pause between each “[API Request]” line and its result, which confirmed the async behavior was working as intended rather than just compiling without error.

Overall Takeaway

This assignment connected three ideas that had previously felt separate: asynchronous control flow with Future/async/await, Dart's compile-time null safety, and exception handling with try/catch. The biggest shift in my understanding was seeing that a well-designed async function signature (Future<UserProfile?>) actually documents its own failure modes: the ? says “this might legitimately come back empty,” and anything more severe is left to surface as a thrown exception instead of being folded into the return type. That distinction is the practical payoff of combining null safety with exceptions rather than relying on just one of them everywhere.